

Archlab 1 Report

By Deng Yufan.

Methodology

Direct implement

In `lab1-c.c`, I write the most common FFT implement. But since there is not STL, I use $X \rightarrow Y \rightarrow X$ instead of `swap`. Since in `data.ans`, all integer is modded into $[0, 2^{31})$, I add a $\text{mod } 2^{31}$ for-loop in the end.

In `lab1-riscv.S`, I simply translate the C language into RISC-V, every sentence in C corresponds to a continuous range of RISC-V instructions. Since I has no experience in writing RISC-V, I didnt do well in allocate virables to registers, and make the code very ugly. When I try to make the code more beautiful, I missed to modify register name in some of instructions, this took a lot time to debug. I also tried to dynamicly calc exp table in inner loop of FFT, but this leads to exponentially increasing precision error, so I gave up this method.

Manual

In `lab1-riscv-manual.S`, I use a more reasonable for-loop implement, use 1 `branch` in loop body instead of 1 `branch` and 1 `jump`. I restructured the main FFT calculation, and it looks much more beautiful. I backed to use `swap` instead of 2 array copy, and move the `reverse` function into the for loop body. I write a Radix-4 (it is my first time to do such optimize since it doesnt work in C++) instead of Radix-2. And I found I don't need to keep $\text{mod } 2^{31}$ for-loop since `data.ans` is adopted to RISC-V at the very beginning.

A fun fact is that when the code write to `data.out`, it won't clear all bits before writing, and if I ran a wrong code before, the code outputs a long text, then after that I run a correct code, there will appear extra integers in the end of `data.out`.

For manual optimize, most things I did, didnt give a big benefit to cycles but decreased #instructions. The most significant two optimization is use Radix-4 and use `swap` instead of 2 copy. Since a algebraic instruction only takes <10 cycle, but memory access (`lw` and `sw`) takes ~200 cycle, so avoid often memory access has great potential to optimize the code.

Use `swap` instead of 2 copy, leads to N load&save instead of $2N$ load&save; use Radix-4, for every size-4 block, we only need to load 4 element in `x`, 3 element in `expTable`, and save 4 element to `x`; but use Radix-2 we need do 4 times of load 2 element in `x`, 1 element in `expTable` and save 2 element to `x`. So this is 11 load/save vs. 20 load/save optimize. Unfortunately, since $N = 128$, we still need to use Radix-2 to end up, so the speedup is less than 2x.

AI

In `lab1-riscv-opt.S`, I communicated with ChatGPT 5.6 Luna Thinking, found out some optimize method, and asked AI to implement the optimization. I mainly used two prompts:

1. 这个 fft 代码现在在 N=128 时需要 215821 cycle, CPI=10.7。如何进一步优化这个代码，使得它的用时减少到 10w cycle。
2. 以我写的 shit8.S 为基准，加入三个优化：pingpong 优化来干掉 reverse 阶段的读写；特判 pass 1 来干掉 expTable 的读取；以及指令重排。你应当基于 riscv 规范来编写代码，不使用 s0-s11,a0-a7,t0-t6 以外的寄存器作为中间变量。你不能拖后定点数的右移到加减法之后，否则会导致精度下降。你应当严格保证代码的正确性，对任何 $N \leq 128$ 且 N 是 2 的幂成立。注意我目前的代码是 Radix-4 加上尾部的一次 Radix-2。你应当保留这个结构。你只能修改 dft_transform_impl 内的代码。你不需要现在输出你写的代码。你应当提前确定好每个部分的寄存器所对应的变量，可以参考我目前的实现，但并不绝对，你可以自己变通。你需要用你的话复述我的需求，并且详细描述每个部分寄存器与变量的对应，以便我确认你的理解正确。

I expected to see a ~1.25x speedup than manual version, since the memory access reduces from ~3.2k to ~2.5k, and maybe permulating the instructions can make CPU running more parallel. But in fact I get a 0.81x slowdown. I asked Gemini 3.6 Flash Thinking to do the same thing, but Gemini did even worse.

Although I only put in 2 prompts, I argued with AI a long time to try to pull them out of illusion. For example, AI think I can fix every `expTable` out of for loop, then I can save a lot load operation.

AI gave a intersting method to reduce memory access. Since we need to do `X[reverse(i)] <- X[i]` before FFT, but note that the first pass of FFT, we have all coefficient to be 1. So we can combine `X[reverse(i)] <- X[i]` and the first pass, as `Y[reverse(j)] <- X[j] + X[j + 1]`, and in incoming pass, we do `X <- Y` and so on. When $N = 128$, and I use Radix-4, there is 4 pass: i=1, i=4, i=16 and i=64. So there will be `X -> Y -> X -> Y -> X` and the final ans will lie in `X`. As a result, we can reduce 448 l/s operations in bit reverse before FFT, and 192 load in first pass to load expTable, make total l/s from ~3.2k to ~2.5k.

Results

Item	C program with <code>-O0</code>	unoptimized RISC-V	Manual RISC-V	AI RISC-V
Cycles	867746	428305	215821	274327
Instructions Retired	128997	31654	20131	17071
CPI	6.73	13.53	10.72	16.07
Speedup	0.49x	1x	1.98x	1.56x

Question Answering

Q1

O3 uses function inlining, which move `reverse` and `mymul` into the FFT function.

And it uses dynamic address calculation, which no longer need to calculate some shift and add operations to get address of some element in an array.

Further more, it uses `memcpy` instead of my `X -> Y -> X` copy.

Q2

I thought of using Radix-8 to further reduce memory load/save, but since registers are only 32, and Radix-8 needs a lot of registers, so I gave up.

The code still has a big cache miss rate, which is the code's bottleneck. Maybe it can be solved by smartly organizing the structure of the array and making good use of L1 cache.

Q3

I tried to use AI to help me write the beginning version of RISC-V, but it used REALLY UGLY (rather than me) variable $\leftrightarrow$ register mapping, and I ended up writing the code myself for further optimizing.

I struggled **2 days** to make AI spawn well-optimized codes, but most of the time AI only optimized #instructions but #cycles. I think AI doesn't know where the bottleneck is and uses some **dead** (a.k.a. 僵硬的) guidelines to optimize.

So I think humans can organize the code's logic better, but AI is more high-speed to produce a ugly but able to run code.

AI didn't do anything helpful to make the code faster. It only reduces the #instructions. So I won't agree that AI is smarter than humans. (or maybe I just wrote bad prompts, used bad models)

AI can translate my C++ code into RISC-V well, but if I tell them to do some optimization on specific RISC-V code, AI doesn't know how to do. So I think AI cannot be very creative, and doesn't have a good taste about whether an optimization is good, or even can't tell whether an optimization is actually working.